

Supervised Learning Part 2

KNN, Bayesian Learning, SVM, Decision Tree, and Random Forest

Created By : Abdan Hafidz

Abstract

This module deconstructs five fundamental algorithms of Supervised Learning: K-Nearest Neighbors, Bayesian Learning, Support Vector Machines, Decision Trees, and Random Forests. Rather than merely listing steps, we explore the mathematical intuition behind the formulas, visualize the decision boundaries, and apply these concepts to tabular data problems. We aim to move beyond "using" libraries to "understanding" the stochastic and geometric engines that drive prediction.

1 Introduction: The Mapping Problem

At its heart, supervised learning is an exercise in function approximation. We are given a set of inputs, which we denote as our feature space X , and a set of corresponding outputs, our label space Y . Our objective is not merely to memorize this pairing, but to discover the hidden function $f : X \rightarrow Y$ that maps features to labels. The challenge lies in the fact that we never see the true f ; we only see a dataset D containing noisy samples generated by f . Our task is to construct a hypothesis h that approximates f well enough to predict y for unseen x . The five algorithms we discuss here represent five distinct philosophical approaches to finding this function: looking at neighbors, calculating probabilities, drawing lines, asking questions, and consulting a crowd.

2 K-Nearest Neighbors (KNN): The Geometry of Similarity

We begin with perhaps the most intuitive algorithm, which rests on the manifold hypothesis: similar things exist in close proximity to one another. K-Nearest Neighbors (KNN) does not "learn" a model in the traditional sense. Instead, it memorizes the training data and postpones the computation until a prediction is requested. This makes it a "lazy learner."

2.1 Mathematical Abstraction: Defining Distance

To find the "nearest" neighbors, we must mathematically define "near." In a 2-dimensional space (like a piece of paper), we intuitively use straight-line distance. If our data points are vectors $x = (x_1, x_2, \dots, x_n)$ and query point $q = (q_1, q_2, \dots, q_n)$, the most common metric is the Euclidean Distance.

Mathematical Abstraction: Euclidean Distance Derivation

The distance $d(x, q)$ is derived from the Pythagorean theorem extended to n -dimensions. We calculate the squared difference in each dimension, sum them, and take the square root to return to the original scale.

$$d(x, q) = \sqrt{\sum_{i=1}^n (x_i - q_i)^2}$$

Here, $(x_i - q_i)$ represents the geometric separation on axis i . We square it to eliminate negative signs (ensuring magnitude matters, not direction).

2.2 Case Study: The Admission Test

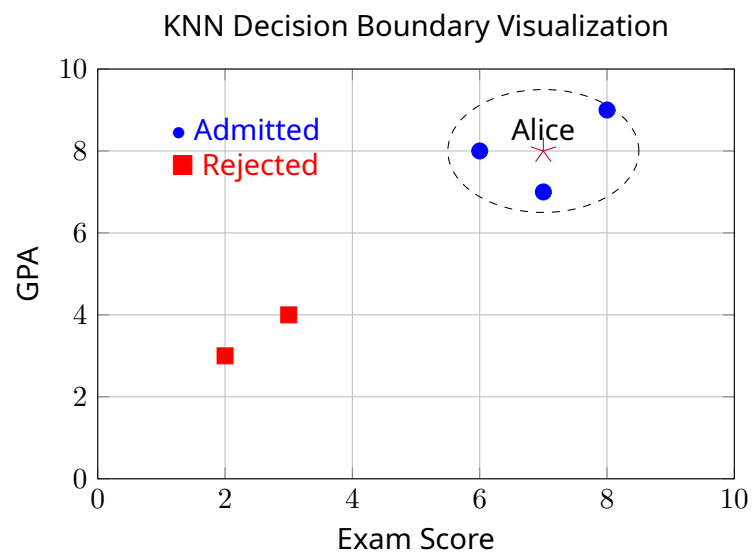
Consider a scenario where we want to predict if a student is admitted (Class 1) or rejected (Class 0) based on two features: Exam Score (x_1) and GPA (x_2). We have a new student, Alice, with Exam Score 7 and GPA 8. We choose $K = 3$.

Student	Exam (x_1)	GPA (x_2)	Admitted (Y)
A	8	9	Yes (1)
B	2	3	No (0)
C	7	7	Yes (1)
D	3	4	No (0)
E	6	8	Yes (1)

To classify Alice ($q = [7, 8]$), we calculate the distance to every known point.

1. ****Distance to A (8, 9):**** $\sqrt{(8-7)^2 + (9-8)^2} = \sqrt{1^2 + 1^2} = \sqrt{2} \approx 1.41$ 2. ****Distance to B (2, 3):**** $\sqrt{(2-7)^2 + (3-8)^2} = \sqrt{(-5)^2 + (-5)^2} = \sqrt{50} \approx 7.07$ 3. ****Distance to C (7, 7):**** $\sqrt{(7-7)^2 + (7-8)^2} = \sqrt{0 + 1} = 1.00$ 4. ****Distance to D (3, 4):**** $\sqrt{(3-7)^2 + (4-8)^2} = \sqrt{16 + 16} = \sqrt{32} \approx 5.65$ 5. ****Distance to E (6, 8):**** $\sqrt{(6-7)^2 + (8-8)^2} = \sqrt{1 + 0} = 1.00$

The sorted distances are: E (1.0), C (1.0), A (1.41), D (5.65), B (7.07). The top 3 nearest neighbors are E, C, and A. Their labels are Yes, Yes, and Yes. **Conclusion:** Since 3 out of 3 neighbors voted "Yes", Alice is predicted to be Admitted.



3 Bayesian Learning: The Architecture of Belief

While KNN looks at geometry, Naive Bayes looks at probability. It is rooted in a fundamental question: *How should our belief about an outcome change when we observe new evidence?*

3.1 Theoretical Framework: Bayes' Theorem

The formula for Naive Bayes is not an arbitrary rule; it is a direct consequence of conditional probability logic. We want to find the probability of a hypothesis H (e.g., "Play Tennis") given some Evidence E (e.g., "Wind is Strong").

Mathematical Abstraction: Bayes' Equation Decomposition

$$P(H|E) = \frac{P(E|H) \cdot P(H)}{P(E)}$$

Let us decode the variables:

- $P(H|E)$ is the **Posterior**: The probability we want to calculate.
- $P(H)$ is the **Prior**: How likely is the hypothesis generally, before looking at data? (e.g., Is it rainy season?)
- $P(E|H)$ is the **Likelihood**: If the hypothesis were true, how likely is this evidence? (e.g., If we play tennis, is it usually windy?)
- $P(E)$ is the **Evidence**: The probability of the evidence occurring at all. In classification, we often ignore this denominator because it is constant across all classes.

Why is it called "Naive"? Because calculating the joint likelihood of many features (e.g., Wind AND Humidity AND Temp) requires a massive amount of data. We make a "naive" assumption that features are *independent* given the class. This allows us to simply multiply probabilities:

$$P(E_1, E_2|H) \approx P(E_1|H) \times P(E_2|H)$$

3.2 Case Study: Weather Prediction

We want to predict if we will "Play Tennis" (Y) based on "Weather" (X).

Day	Weather	Play?
1	Sunny	No
2	Overcast	Yes
3	Rainy	Yes
4	Sunny	Yes
5	Sunny	No
6	Rainy	No
7	Overcast	Yes
8	Rainy	No

Problem: Today is **Sunny**. Should we play?

Step 1: Calculate Priors (General probabilities)

Total days: 8. Days Yes: 4. Days No: 4.

$$P(Yes) = 4/8 = 0.5$$

$$P(No) = 4/8 = 0.5$$

Step 2: Calculate Likelihoods (Evidence probability)

We need $P(Sunny|Yes)$ and $P(Sunny|No)$.

- Out of 4 "Yes" days, how many were Sunny? Only Day 4. So, $P(\text{Sunny}|\text{Yes}) = 1/4 = 0.25$.
- Out of 4 "No" days, how many were Sunny? Day 1 and Day 5. So, $P(\text{Sunny}|\text{No}) = 2/4 = 0.50$.

Step 3: Calculate Posterior (Unnormalized)

$$\text{Score(Yes)} = P(\text{Sunny}|\text{Yes}) \times P(\text{Yes}) = 0.25 \times 0.5 = 0.125$$

$$\text{Score(No)} = P(\text{Sunny}|\text{No}) \times P(\text{No}) = 0.50 \times 0.5 = 0.250$$

Since $0.250 > 0.125$, the model predicts **No Play**. The math shows that Sunny weather is strongly correlated with not playing in this specific dataset.

4 Support Vector Machines (SVM): The Widest Street

If KNN is a localized voter and Bayes is a statistician, SVM is a geometer. Its goal is simple yet profound: separate the classes not just by *any* line, but by the *best* line. The “best” line is defined as the one that maximizes the **margin**—the empty space between the classes.

4.1 Mathematical Abstraction: Hyperplanes and Margins

A line in 2D (or a hyperplane in nD) is defined by the equation $w \cdot x + b = 0$, where w is the normal vector perpendicular to the line, and b is the bias (offset).

For any point x , the value $w \cdot x + b$ tells us which side of the line the point is on. If the value is positive, we classify as Class 1. If negative, Class -1.

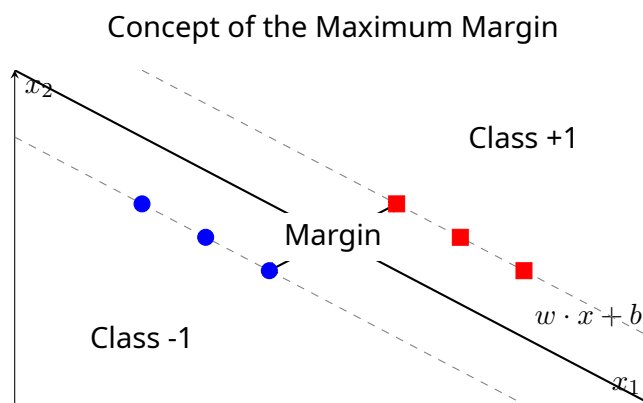
We want to constrain our system such that for the nearest points (the support vectors), the value is exactly 1 or -1.

$$w \cdot x_+ + b = 1 \quad \text{and} \quad w \cdot x_- + b = -1$$

The width of the margin can be mathematically derived as:

$$\text{Margin Width} = \frac{2}{\|w\|}$$

To maximize the margin, we must *minimize* $\|w\|$. This turns machine learning into a constrained optimization problem.



4.2 The Kernel Trick: Warping Space

What if the data cannot be separated by a straight line (e.g., a red circle inside a blue ring)? SVM handles this using the **Kernel Trick**. We project the data into a higher dimension.

Imagine 2D data on a flat sheet of paper. You cannot draw a straight line to separate the inner circle from the outer ring. But if you lift the inner circle up (adding a 3rd dimension based on distance from center), you can slide a flat sheet (a plane) between them. When you project this back down to 2D, the straight plane looks like a curved boundary. This is the power of SVM.

5 Decision Trees: The Game of 20 Questions

Decision Trees mimic human logical reasoning. When a doctor diagnoses a patient, they don't solve a dot product equation. They ask a sequence of filtering questions: "Does the patient have a fever?" If yes, check throat. If no, check stomach.

5.1 Mathematical Abstraction: Entropy and Information Gain

The core problem in building a tree is deciding which question to ask first. We want the question that reduces uncertainty the most. We measure uncertainty using a concept from Thermodynamics and Information Theory called **Entropy**.

Mathematical Abstraction: Shannon Entropy (H)

$$H(S) = - \sum_{i=1}^c p_i \log_2(p_i)$$

Where p_i is the probability (proportion) of class i in the dataset S .

- If a bucket is 100% Red Balls, $p_{red} = 1$. $\log_2(1) = 0$. Entropy is 0 (Pure, no surprise).
- If a bucket is 50% Red and 50% Blue, $p = 0.5$. Entropy is 1 (Maximum impurity/chaos).

When we split a dataset on a feature A , we calculate the **Information Gain (IG)**:

$$IG(S, A) = H(S) - H(S|A)$$

This translates to: *The uncertainty I had before* minus *the weighted average uncertainty I have after the split*. We always choose the feature with the highest Information Gain.

5.2 Case Study: Job Offer Acceptance

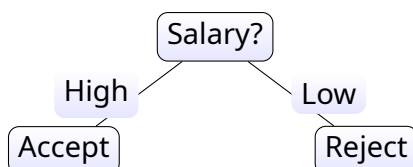
Imagine deciding to accept a job based on Salary (High/Low) and Commute (Long/Short). Data: 1. High Salary, Short Commute → Accept 2. High Salary, Long Commute → Accept 3. Low Salary, Short Commute → Reject 4. Low Salary, Long Commute → Reject

Entropy of Parent: 2 Accepts, 2 Rejects. 50/50 split. $H(S) = 1.0$.

Option A: Split by Commute? - Short Commute branch: 1 Accept, 1 Reject. (Still messy, Entropy = 1.0). - Long Commute branch: 1 Accept, 1 Reject. (Still messy, Entropy = 1.0). - Information Gain = $1.0 - 1.0 = 0$. This is a useless question.

Option B: Split by Salary? - High Salary branch: 2 Accepts, 0 Rejects. (Pure! Entropy = 0). - Low Salary branch: 0 Accepts, 2 Rejects. (Pure! Entropy = 0). - Information Gain = $1.0 - 0 = 1.0$. This is a perfect question.

The Tree chooses "Salary" as the root node.



5.3 Alternative Metric: Gini Impurity

While Entropy is derived from Information Theory, **Gini Impurity** is a metric often used by the CART (Classification and Regression Trees) algorithm for its computational efficiency (avoiding

logarithmic calculations). It measures the likelihood of incorrect classification of a new instance of a random variable, if that new instance were randomly classified according to the distribution of class labels in the dataset.

Mathematical Abstraction: Gini Impurity Formula

$$Gini(S) = 1 - \sum_{i=1}^c (p_i)^2$$

Where p_i is the probability of an item belonging to class i .

- **Maximum Impurity:** In a binary class (50/50 split), $1 - (0.5^2 + 0.5^2) = 0.5$.
- **Minimum Impurity:** If 100% of samples are one class, $1 - (1^2) = 0$.

5.4 Case Study: Credit Approval (Gini Calculation)

Let us manually trace a Gini calculation to determine the best root node for a Credit Approval dataset with 5 applicants. **Goal:** Predict if Credit is **Approved (A)** or **Rejected (R)**.

Dataset Overview:

- Total Applicants: 5
- Class Counts: 3 Approved (A), 2 Rejected (R).

Step 1: Calculate Parent Gini

$$Gini(Parent) = 1 - \left(\left(\frac{3}{5} \right)^2 + \left(\frac{2}{5} \right)^2 \right) = 1 - (0.36 + 0.16) = 1 - 0.52 = \mathbf{0.48}$$

Step 2: Evaluate Split by Feature "Credit Score"

Assume we split by Credit Score into two branches: Good and Bad.

Branch 1: Good Credit (3 People)

- Distribution: 3 Approved, 0 Rejected. (Pure node)
- $Gini(Good) = 1 - (1^2 + 0^2) = \mathbf{0.0}$

Branch 2: Bad Credit (2 People)

- Distribution: 0 Approved, 2 Rejected. (Pure node)
- $Gini(Bad) = 1 - (0^2 + 1^2) = \mathbf{0.0}$

Step 3: Calculate Weighted Gini of the Split

We calculate the weighted average of the impurities based on the number of samples in each branch.

$$Gini_{Split} = \frac{3}{5}Gini(Good) + \frac{2}{5}Gini(Bad)$$

$$Gini_{Split} = 0.6(0) + 0.4(0) = \mathbf{0.0}$$

Comparison: The Gini Impurity dropped from 0.48 (Parent) to 0.0 (Split). This reduction of 0.48 represents a "perfect split," indicating that Credit Score is an ideal feature to categorize this specific dataset.

6 Random Forest: The Wisdom of the Crowd

Decision Trees have a fatal flaw: they memorize data too well. If you have a noisy outlier, a single Decision Tree will create a specific complex rule just to explain that one anomaly. This is called **Overfitting**.

Random Forest solves this by creating an ensemble (a group) of trees. It relies on the statistical concept that a large number of uncorrelated errors average out to zero.

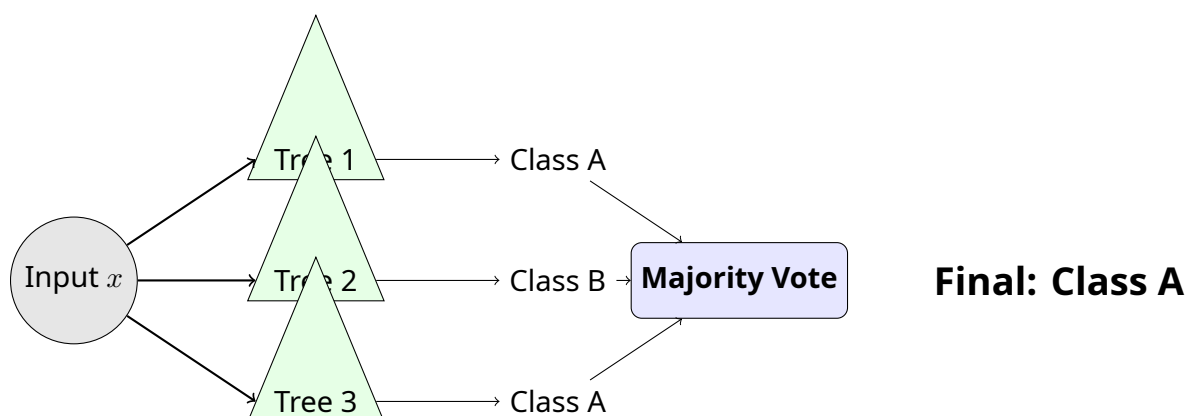
6.1 The Mechanism: Bagging and Feature Randomness

A Random Forest does not just build 100 identical trees. That would be useless. It enforces diversity through two methods:

1. **Bagging (Bootstrap Aggregating):** Each tree is trained on a random subset of the data sampled *with replacement*. Tree A might see Student X but not Student Y. Tree B might see Student Y twice but not Student X. This ensures the trees learn different patterns.
2. **Random Feature Selection:** At every split in the tree, the algorithm is not allowed to consider *all* features. It is forced to pick the best feature from a random subset. This prevents one dominant feature (like "Income" in a loan dataset) from making every tree look identical.

6.2 Visualization of the Voting Process

Imagine we have a new data point. We pass it through 3 trees in our forest.



Tree 1 and Tree 3 found patterns suggesting Class A. Tree 2 was perhaps misled by noise and predicted Class B. The Forest aggregates these votes and correctly outputs Class A. By averaging the results, the variance (the jitteriness of the model) is drastically reduced, creating a model that is robust, stable, and accurate.

7 Summary

We have traversed the landscape of supervised learning. We saw how **KNN** relies on local memory, **Naive Bayes** on global probability statistics, **SVM** on geometric optimization, **Decision Trees** on hierarchical logic, and **Random Forest** on democratic consensus. Understanding these mathematical underpinnings allows us to choose the right tool not by guessing, but by analyzing the structure of our data.